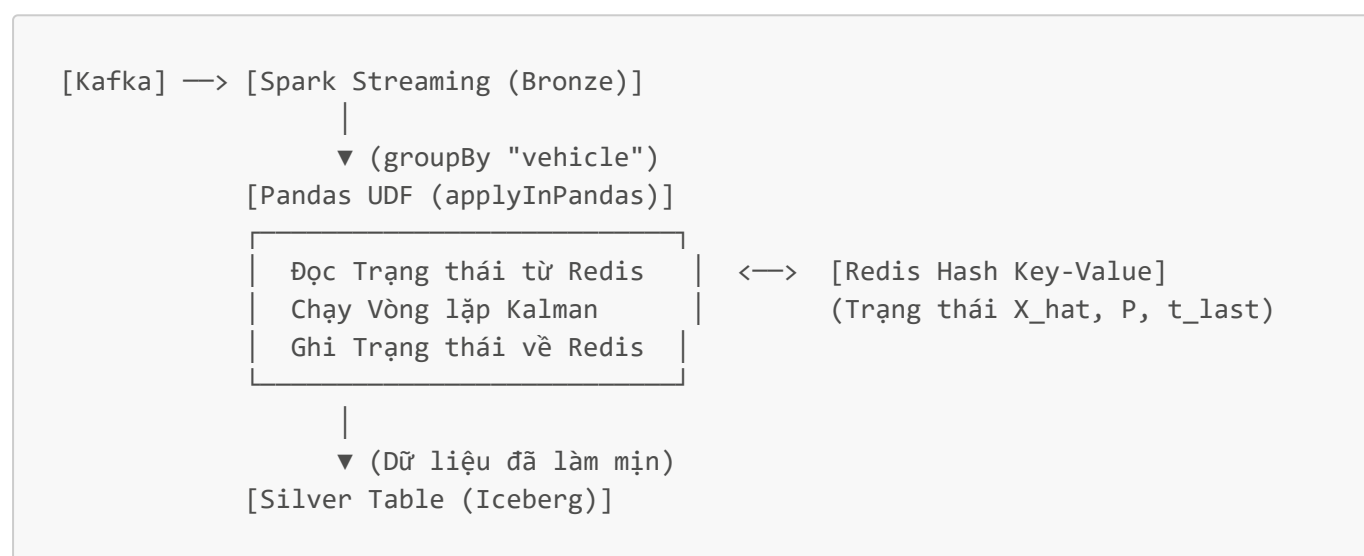


Hướng dẫn Triển khai Kalman Filter & So sánh Kiến trúc (Redis vs Non-Redis)

Tài liệu này trình bày phương pháp triển khai kỹ thuật bộ lọc Kalman Filter trên hệ thống dữ liệu lớn Spark Streaming tại tầng Silver, đồng thời so sánh chi tiết giữa việc sử dụng tầng lưu trữ trạng thái ngoài (Redis) so với các phương pháp truyền thống.

1. Kiến trúc Triển khai Hệ thống (Spark Streaming & Pandas UDF)

Bộ lọc Kalman Filter được tích hợp trực tiếp vào dòng xử lý dữ liệu streaming thời gian thực từ Bronze lên Silver:



Chi tiết luồng thực thi:

- Dòng dữ liệu từ Bronze được gom nhóm theo phương tiện (**vehicle**).
- Cụm Spark phân phối các nhóm phương tiện này cho các Worker thông qua hàm **applyInPandas** (Pandas UDF).
- Tại mỗi Worker, class **RedisBackedKalmanFilter** khởi tạo kết nối đến Redis, nạp trạng thái lọc trước đó của xe, thực hiện làm mịn chuỗi điểm định vị hiện tại của micro-batch, và lưu lại trạng thái cuối cùng về Redis trước khi kết thúc batch.

2. Thách thức về Trạng thái (The Stateful Challenge)

Kalman Filter là một thuật toán **Stateful (yêu cầu trạng thái liên tục)**. Để tính toán làm mịn cho điểm GPS ở giây thứ t , nó bắt buộc phải biết trạng thái ước lượng tối ưu trước đó ($\hat{\mathbf{x}}_{t-1}$), ma trận hiệp phương sai sai số (P_{t-1}), và mốc thời gian nhận điểm cuối cùng (t_{last}).

Trong xử lý luồng dữ liệu lớn (Streaming):

- Dữ liệu đổ về theo từng micro-batch (ví dụ: mỗi 10 giây).

- Các điểm định vị của cùng một phương tiện sẽ bị chia cắt ngẫu nhiên qua các ranh giới micro-batch khác nhau.
- Nếu không lưu giữ trạng thái lịch sử, mỗi micro-batch sẽ phải chạy bộ lọc độc lập từ đầu (vận tốc bắt đầu từ 0, ma trận P chưa hội tụ), khiến bộ lọc mất đi khả năng làm mịn xuyên suốt chuỗi chuyển động.

3. Bảng So sánh Kiến trúc: Có Redis vs Không Redis

Để giải quyết bài toán Stateful trong Spark Streaming, có 3 hướng tiếp cận chính. Dưới đây là bảng so sánh chi tiết:

Tiêu chí	Phương án 1: Dùng Redis (Hiện tại)	Phương án 2: Spark <code>mapGroupsWithState</code> (Native)	Phương án 3: Lưu lịch sử cục bộ (Window Join)
Kiến trúc lưu trữ	Trạng thái lưu trữ ngoài (Externalized State) trên Redis Cache.	Trạng thái lưu trữ trực tiếp trong RAM của Spark Executor.	Tự join/quét ngược bảng Silver trong mỗi micro-batch để lấy điểm cũ.
Tải bộ nhớ (Memory Footprint)	Cực kỳ nhẹ. Worker chỉ cần giữ dữ liệu của micro-batch hiện tại ($\approx$ vài KB).	Nặng. Executor phải giữ trạng thái của toàn bộ hàng ngàn xe trong RAM để so sánh.	Cực kỳ nặng và phình to. Tăng theo cấp số nhân khi lịch sử chạy trong ngày tăng lên.
Độ trễ (Latency)	Rất thấp (< 1ms). Đọc/ghi Redis thông qua Redis Hash có tốc độ phản hồi cực nhanh.	Thấp. Truy cập RAM cục bộ nhanh nhưng bị ảnh hưởng bởi Garbage Collection của Java JVM.	Rất cao. Quá tải do phải thực hiện các truy vấn đọc đĩa đề lên bảng Iceberg liên tục.
Khả năng Mở rộng (Scalability)	Tối ưu. Các Spark Executor hoàn toàn không giữ trạng thái (Stateless), dễ dàng scale-out/scale-in hoặc phục hồi khi gặp sự cố.	Khó khăn. Khi scale-out, Spark phải phân phối lại trạng thái (State Rebalancing) giữa các node, gây nghẽn mạng.	Không thể mở rộng. Hệ thống sẽ sập khi lượng xe hoặc số ngày chạy tăng cao.
Độ tin cậy & Phục hồi (Fault Tolerance)	Rất cao. Nếu Spark Executor bị chết giữa chừng, executor mới lên chỉ cần đọc lại trạng thái từ Redis và tiếp tục chạy mà không mất dữ liệu lịch sử.	Trung bình. Phụ thuộc vào Spark Checkpoint. Việc phục hồi từ checkpoint lớn tốn nhiều phút.	Thấp. Rất dễ xảy ra lỗi mất nhất quán dữ liệu do overlap giữa các batch ghi đĩa.
Truy cập chéo (Cross-System Sharing)	Tuyệt vời. Các ứng dụng bên ngoài (Dashboard giám sát, API thời gian thực, Web	Không thể. Dữ liệu trạng thái bị đóng kín bên trong JVM của cụm Spark.	Trung bình. Phải thực hiện câu lệnh SQL truy vấn vào

Tiêu chí	Phương án 1: Dùng Redis (Hiện tại)	Phương án 2: Spark mapGroupsWithState (Native)	Phương án 3: Lưu lịch sử cục bộ (Window Join)
	bản đồ) có thể đọc trực tiếp trạng thái xe hiện tại từ Redis.		database (độ trễ cao).
Dọn dẹp bộ nhớ (GC / TTL)	Tự động. Sử dụng cơ chế TTL của Redis (tự động xóa trạng thái xe sau 2 giờ không hoạt động).	Phức tạp. Phải tự lập trình cơ chế timeout để giải phóng trạng thái cũ trong code.	Phức tạp. Phải chạy job dọn dẹp hoặc partition cũ định kỳ.

4. Chi tiết Thiết kế Trạng thái trên Redis (Redis Key Design)

Hệ thống sử dụng cấu trúc **Redis Hash** để lưu trạng thái của từng xe:

- **Key Format:** `kalman_state:{vehicle_id}`
- **TTL (Time To Live):** `7200 giây (2 giờ)`. Tự động gia hạn (expire) sau mỗi lần cập nhật.
- **Fields trong Hash:**
 - `x_hat_0, x_hat_1`: Tọa độ vị trí tối ưu (x, y).
 - `x_hat_2, x_hat_3`: Vận tốc ước lượng tối ưu (v_x, v_y).
 - `P`: Hiệp phương sai số 4x4 (được chuyển đổi sang định dạng JSON String).
 - `timestamp`: Thời điểm ghi nhận điểm GPS cuối cùng (ISO Format).

Minh họa Đọc / Ghi Trạng thái (Python):

```
# Đọc trạng thái từ Redis
state = redis_client.hgetall(f"kalman_state:{vehicle}")
if state:
    x_hat = np.array([[float(state["x_hat_0"])], ...])
    P = np.array(json.loads(state["P"]))
    last_timestamp = pd.to_datetime(state["timestamp"])

# Ghi trạng thái về Redis sau khi lọc
new_state = {
    "x_hat_0": str(x_hat[0, 0]),
    "P": json.dumps(P.tolist()),
    "timestamp": timestamp.isoformat()
}
redis_client.hset(f"kalman_state:{vehicle}", mapping=new_state)
redis_client.expire(f"kalman_state:{vehicle}", 7200)
```

5. Kết luận Kỹ thuật

Triển khai bộ lọc **Kalman Filter kết hợp Redis Serving Layer** là một mô hình kiến trúc hiện đại (Externalized State Pattern). Phương án này giúp cô lập logic xử lý tính toán của Spark (luôn giữ trạng thái stateless gọn nhẹ) và đẩy toàn bộ việc lưu trữ trạng thái lưu động sang Redis. Đây là yếu tố cốt lõi giúp hệ thống duy trì độ trễ cực thấp ổn định và dễ dàng mở rộng quy mô phục vụ hàng vạn phương tiện chạy đồng thời trên thực tế.